Conceptos Avanzados de Lenguajes de Programación

Trabajo Practico Obligatorio 1

Alumno: Iovaldi, Sebastián Ángel
Legajo: fai-3482

```
procedure Subprogramas_Ada is

   type Estudiante is record
       Id : Integer;
       Nombre : String (1..20);
       Promedio : Float;
   end record;

   numeros : array (1..5) of Integer := (10, 20, 30, 40, 50);
   estudiante : Estudiante;
   ...
```

estudiante.Id	?
estudiante.Nombre[20]	?
estudiante.Nombre[...]	?
estudiante.Nombre[3]	?
estudiante.Nombre[2]	?
estudiante.Nombre[1]	?
estudiante.Promedio	?
numeros[5]	50
numeros[4]	40
numeros[3]	30
numeros[2]	20
numeros[1]	10
return to _init_	

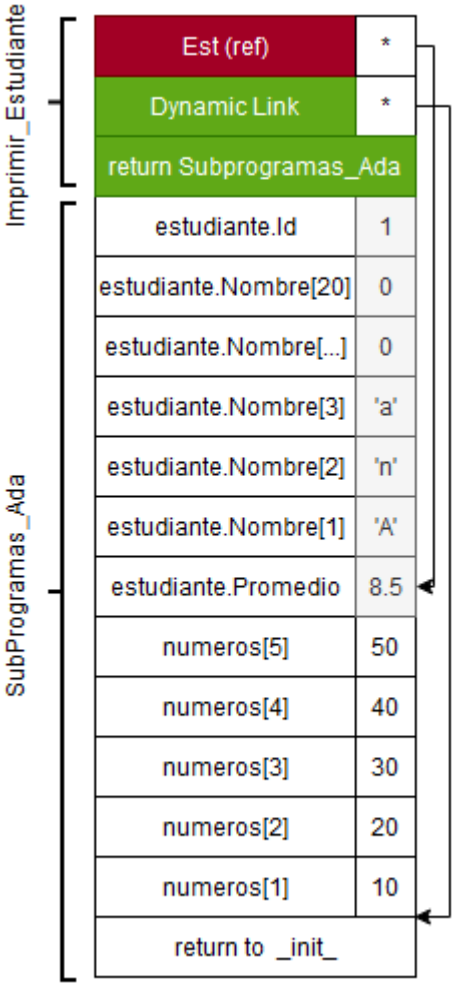
```
begin
    -- Inicialización del estudiante
    estudiante.Id := 1;
    estudiante.Nombre := "Ana";
    estudiante.Promedio := 8.5;
```

estudiante.Id	1
estudiante.Nombre[20]	0
estudiante.Nombre[...]	0
estudiante.Nombre[3]	'a'
estudiante.Nombre[2]	'n'
estudiante.Nombre[1]	'A'
estudiante.Promedio	8.5
numeros[5]	50
numeros[4]	40
numeros[3]	30
numeros[2]	20
numeros[1]	10
return to _init_	

```
...
Imprimir_Estudiante (estudiante);
```

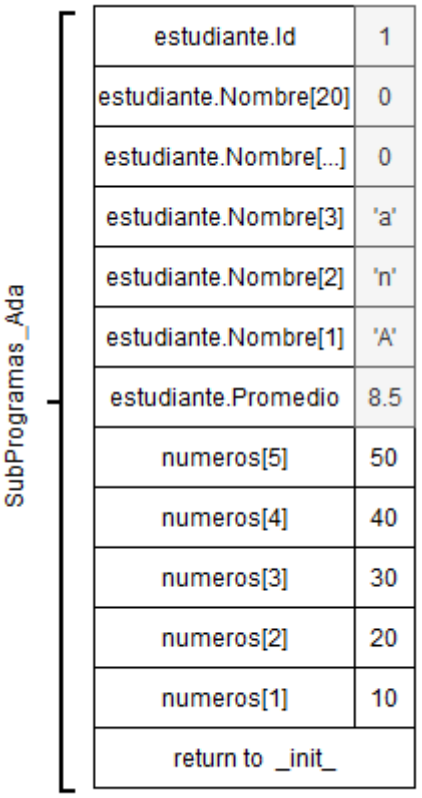
Asumimos pasaje por referencia, Est es solo lectura y tiene un string de 20 caracteres.

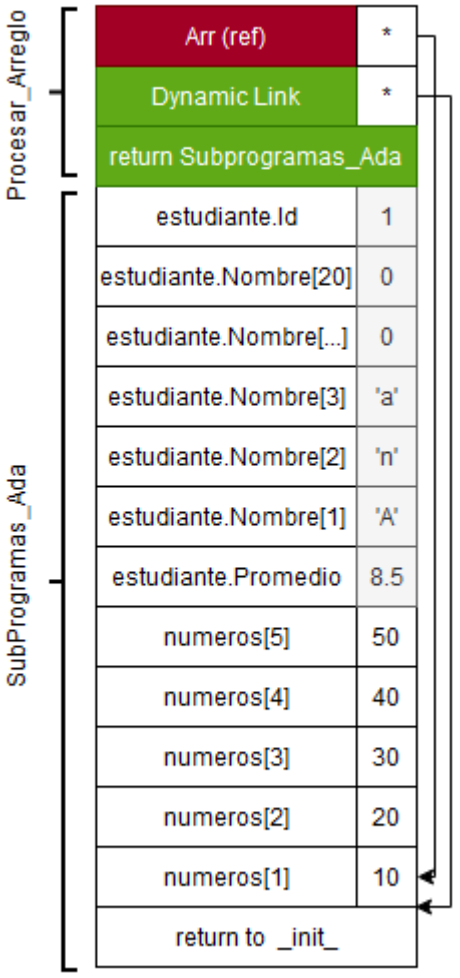
```
procedure Imprimir_Estudiante (Est : in Estudiante) is
begin
    Put_Line ("ID: " & Integer'Image (Est.Id));
    Put_Line ("Nombre: " & Est.Nombre);
    Put ("Promedio: ");
    Put (Est.Promedio, Fore => 1, Aft => 2, Exp => 0);
    New_Line;
  end Imprimir_Estudiante;
```



Salimos de la subrutina...

```
Imprimir_Estudiante (estudiante);
->  ...
```






```
...
   Procesar_Arreglo (numeros);
```

Pasaje por referencia de numeros

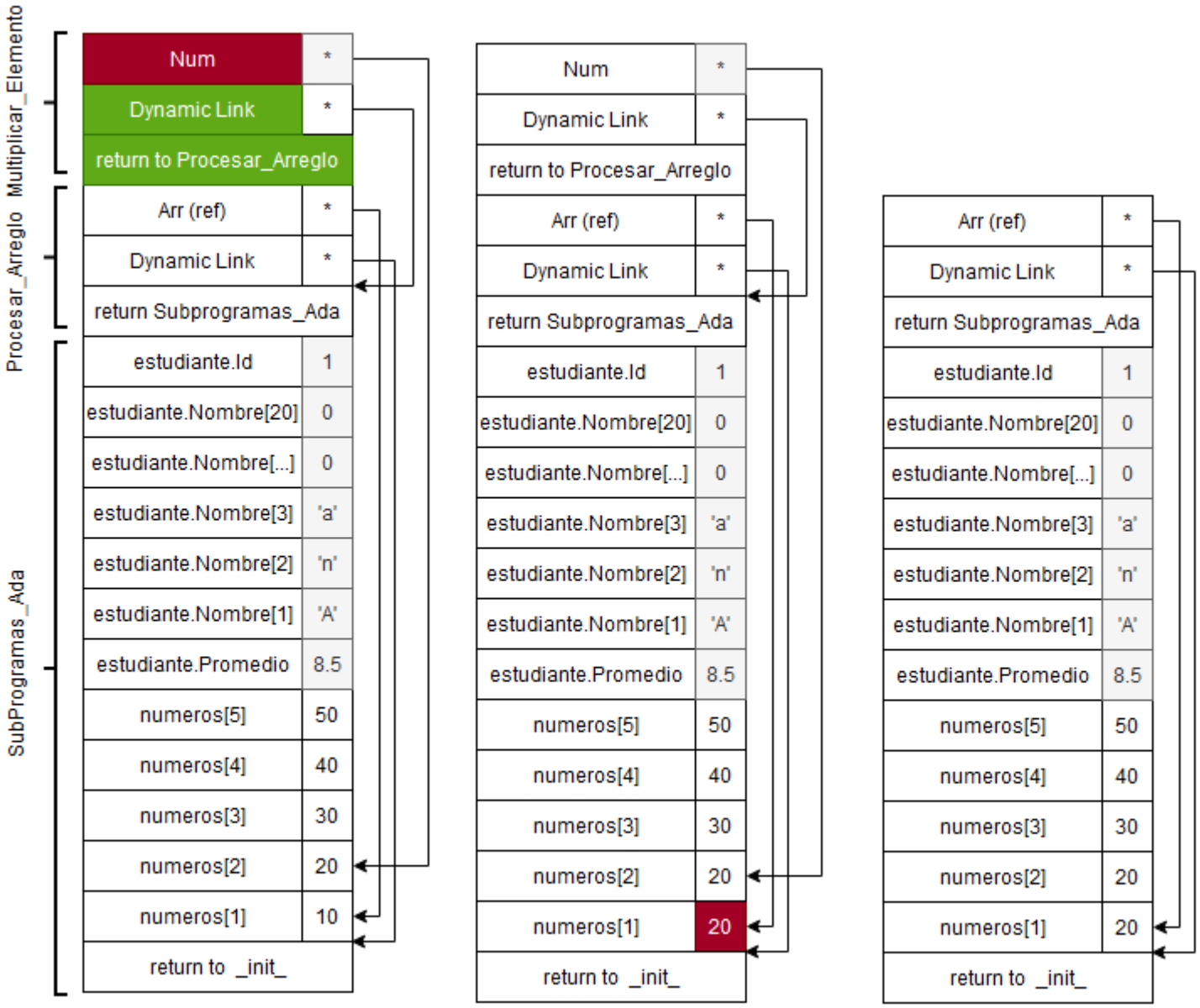
```
procedure Procesar_Arreglo (Arr : in out array (1..5) of Integer
        -- Procedimiento local para multiplicar elementos
            ....
```

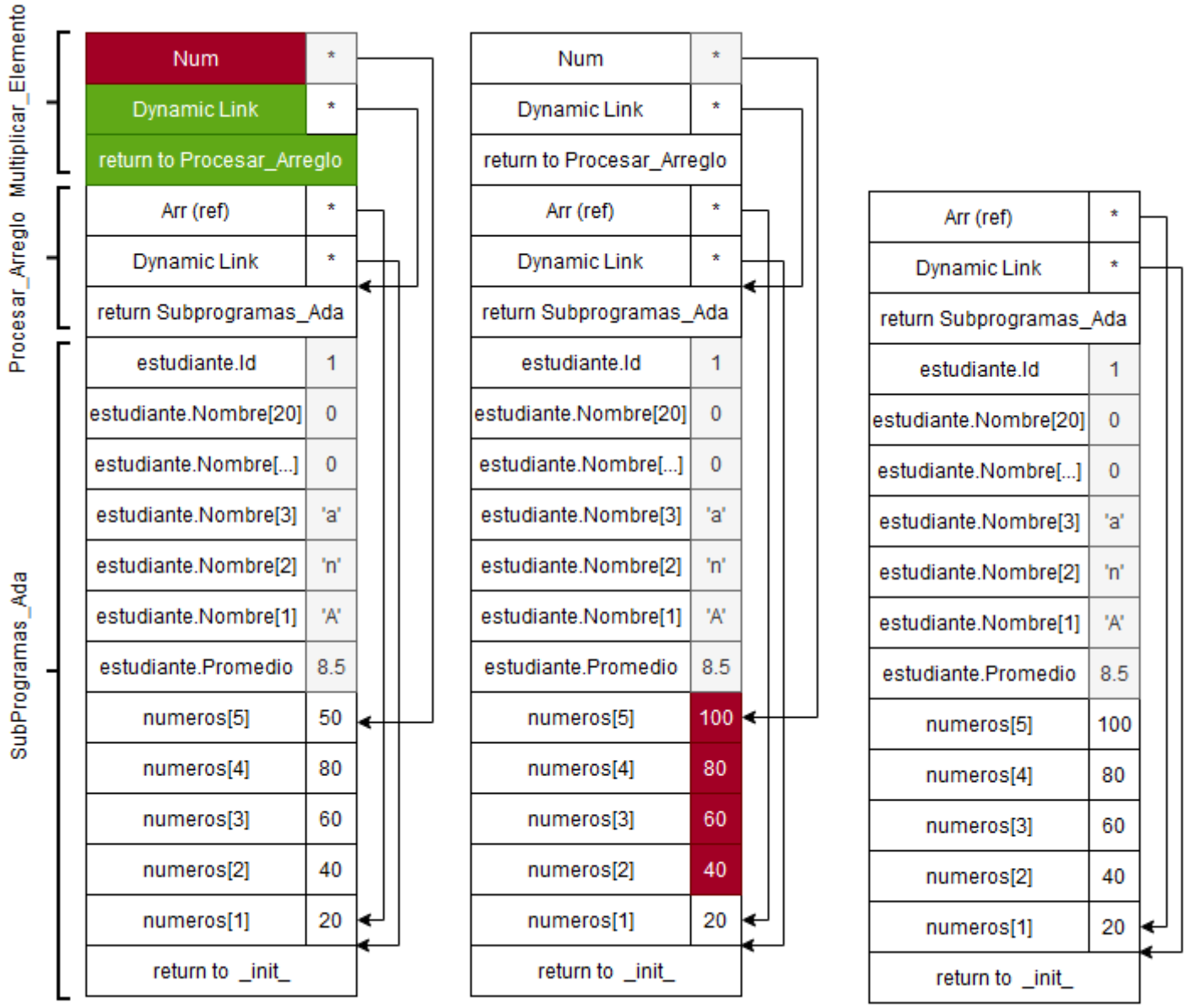
Se llama n veces a Multiplicar_Elemento, se el valor n del elemento por referencia cada iteración

```
      ...
  for I in Arr'Range loop
      Multiplicar_Elemento (Arr(I));
   end loop;
```

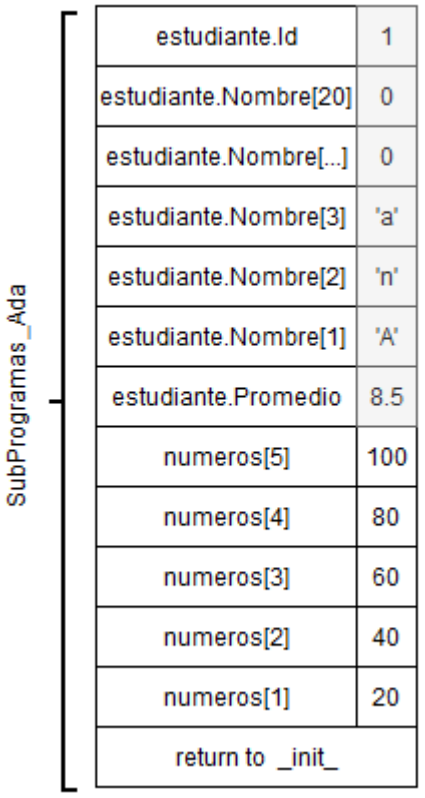
```
procedure Multiplicar_Elemento (Num : in out Integer) is
   begin
       Num := Num * 2;
   end Multiplicar_Elemento;
```

Cinco veces lo mismo.



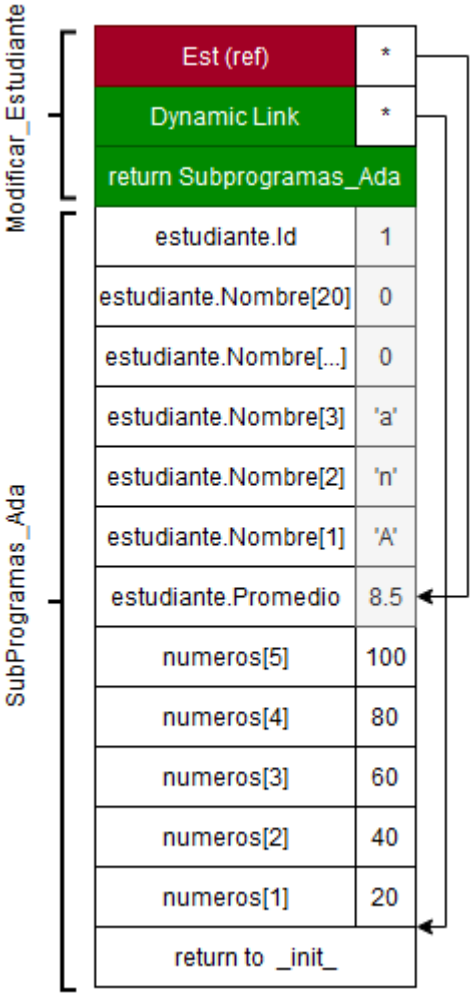

Multiplicar_Elemento
Procesar_Arreglo
SubProgramas_Ada
Num
*
Dynamic Link
*
return to Procesar_Arreglo
Arr (ref)
*
Dynamic Link
*
return Subprogramas_Ada
estudiante.Id
1
estudiante.Nombre[20]
0
estudiante.Nombre[...]
0
estudiante.Nombre[3]
'a'
estudiante.Nombre[2]
'n'
estudiante.Nombre[1]
'A'
estudiante.Promedio
8.5
numeros[5]
50
numeros[4]
80
numeros[3]
60
numeros[2]
40
numeros[1]
20
return to _init_
Num
*
Dynamic Link
*
return to Procesar_Arreglo
Arr (ref)
*
Dynamic Link
*
return Subprogramas_Ada
estudiante.Id
1
estudiante.Nombre[20]
0
estudiante.Nombre[...]
0
estudiante.Nombre[3]
'a'
estudiante.Nombre[2]
'n'
estudiante.Nombre[1]
'A'
estudiante.Promedio
8.5
numeros[5]
100
numeros[4]
80
numeros[3]
60
numeros[2]
40
numeros[1]
20
return to _init_
Arr (ref)
*
Dynamic Link
*
return Subprogramas_Ada
estudiante.Id
1
estudiante.Nombre[20]
0
estudiante.Nombre[...]
0
estudiante.Nombre[3]
'a'
estudiante.Nombre[2]
'n'
estudiante.Nombre[1]
'A'
estudiante.Promedio
8.5
numeros[5]
100
numeros[4]
80
numeros[3]
60
numeros[2]
40
numeros[1]
20
return to _init_

```
Procesar_Arreglo (numeros);
-> ....
```

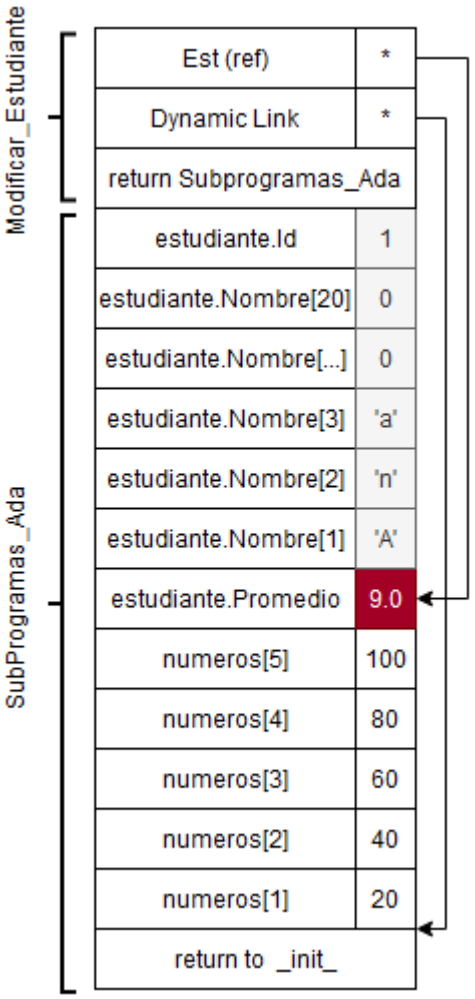



```
Modificar_Estudiante (estudiante);
```

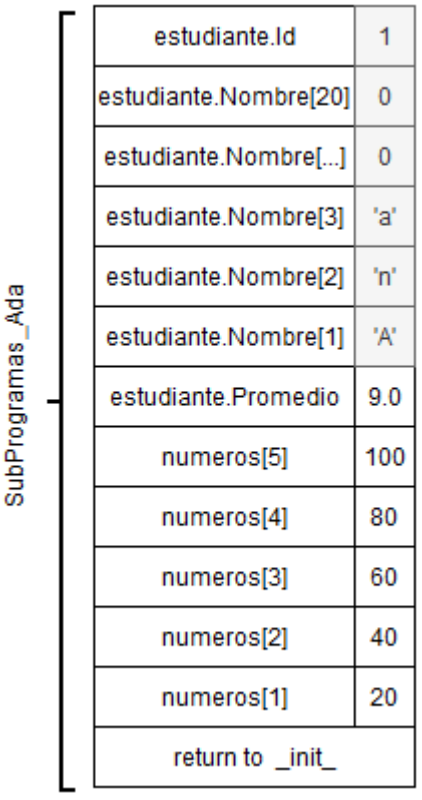
```
-- Procedimiento global para modificar un estudiante
procedure Modificar_Estudiante (Est : in out Estudiante) is
begin
```



```
        Est.Promedio := 9.0;
        Put_Line ("Saliendo de Modificar_Estudiante");
    end Modificar_Estudiante;
```

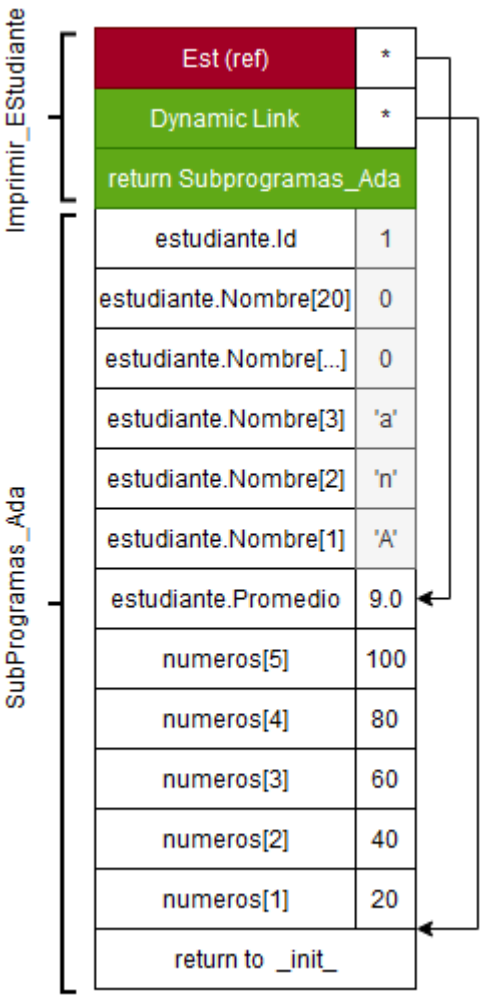



```
        Modificar_Estudiante (estudiante);
    -> ...
        Put_Line ("Arreglo modificado: ");
        for I in numeros'Range loop
            Put (numeros(I), Width => 3);
        end loop;
        New_Line;
        Put_Line ("Estudiante modificado:");
```

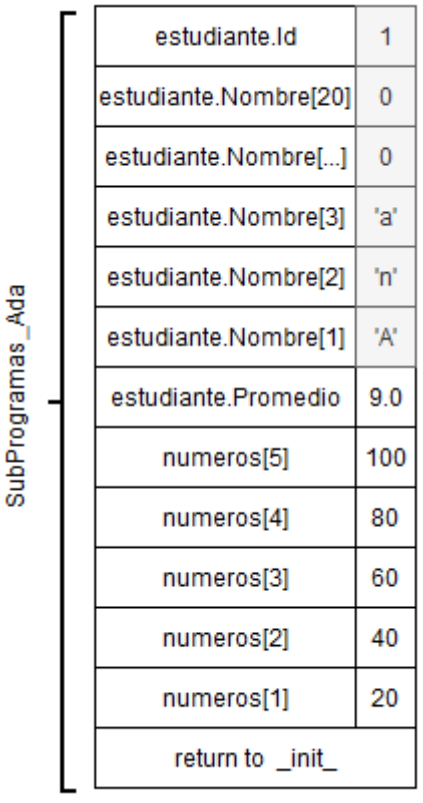


```
        --> Imprimir_Estudiante (estudiante);
```

```
procedure Imprimir_Estudiante (Est : in Estudiante) is
begin
    Put_Line ("ID: " & Integer'Image (Est.Id));
    Put_Line ("Nombre: " & Est.Nombre);
    Put ("Promedio: ");
    Put (Est.Promedio, Fore => 1, Aft => 2, Exp => 0);
    New_Line;
end Imprimir_Estudiante;
```




```
        Imprimir_Estudiante (estudiante);
         -> ...
    end Subprogramas_Ada;
```



```
end Subprogramas_Ada;
```

Scope/Alcance

Se dice del alcance de una variable o función, a la visibilidad que se tiene de la misma en diferentes partes de un programa, y determina el acceso de las mismas dependiendo del contexto.

Static Scope/Alcance Estático

En lenguajes con alcance estático, la visiblidad de las variables se determina antes de iniciar su ejecución, basado en la estructura del código, en tiempo de compilación.

Los símbolos referenciados son buscado en el cuerpo de la subrutina actual y luego en las siguientes, subiendo por la jerarquía de subrutinas anidadas.

Es el tipo de alcance adoptado por la mayoría de los lenguajes.

Dynamic scope/Alcance Dinámico

A diferencia del alcance estático, el dinámico determina la visibilidad de las variables desde las subrutinas, dependiendo del orden y momento en el que son invocadas, y no en la estructura del código.

Una subrutina puede ser llamada en múltiples puntos de un mismo programa, por lo que debe esperarse a la ejecución del mismo para determinar la visibilidad de las variables referenciadas.

La forma de determinar la búsqueda de los símbolos empieza por el cuerpo de la subrutina actual, pero luego se recorren las subrutinas invocadas en orden inverso, siendo el scope más cercano, la ultima subrutina llamda. En términos prácticos, es seguir el segmento de pila del programa en ejecución, hasta dar con la primera ocurrencia de la variable referenciada.

Binding/Vinculación

Hace referencia al momento de vinculación de las variables libres de una subrutina, mediante la visibilidad de las mismas.

Esto es, por ejemplo, cuando una subrutina puede ser pasada por parámetro obteniendo un alias dentro de una segunda subrutina, la determinación del scope de las variables, explicada previamente, puede ocurrir de distintas maneras.

Deep Binding/Vinculación Profunda

En este caso, la visibilidad (scope) con la que se buscan variables libres de una subrutina, es determinado en el contexto donde se declaró dicha subrutina. Como no depende del momento o el contexto donde la subrutina fue invocada, es posible determinar la vinculación en tiempo de compilación.

Shallow Binding/Vinculación Superficial.

Caso contrario, aquí las variables libres son vinculadas en el momento en el que se invocan las subrutinas, en tiempo de ejecución, y las vinculaciones toman en cuenta el contexto actual de la llamada.

Mi lenguaje favorito.

```cpp
#include <iostream>

template<class T> void realizar_tarea(T tarea) // genérico T para pasar subrutin
{
    int sueño = 5;
    int diversion = 5;

    // invoca la subrutina pasada por parametro
    tarea();
}

int main()
{
    int sueño = 0;
    int diversion = 0;

    /*definición funcion lambda (inline)*/
    auto calp = [&]()
    {
        diversion+=99;
        std::cout << "Diversión: " << diversion << "\n";
    };

    auto dalg = [&]()
    {
        sueño+=99;
        std::cout << "Sueño: " << sueño << "\n";
    };

    std::cout << "Realizando tarea de dalg... \n";
    realizar_tarea(dalg);

    std::cout << "Realizando tarea de calp... \n";
    realizar_tarea(calp);

    return 0;
}
```

C++ tiene Static Scope y Deep Binding.

Se declaran las funciones “dalg” y “calp”, anidadas a “main”. Realizando un análisis estático, puede verse que las variables diversion y sueño son encontradas en la subrutina “main”, inicializadas en cero. El binding ocurre en este punto, luego el scope de las variables también ya está determinado.

Se pasan ambas subrutinas por parámetro. Aunque el nuevo contexto tiene sus propias variables, “diversión” y “sueño”, inicializadas en 5, la salida del programa indica que 99+0=99, ignorando las variables locales de “realizar_tarea”

```
seba@desktop:~/fai/calp/tpo01$ g++ main.cpp && ./a.out
Realizando tarea de dalg...
Sueño: 99
Realizando tarea de calp...
Diversión: 99
```

En el hipotético caso, donde el lenguaje tenga Dynamic Scope y Shallow Binding, aún en el punto en donde se declaran las subrutinas “calp” y “dalg” (compilación), todavía no se han vinculado las variables libres.

Dentro del contexto de “realizar_tarea”, se llama a “tarea()”. Como el lenguaje usa Shallow Binding, es en este momento donde ocurre la vinculación de las variables libres.

Luego la visibilidad de las variables, por utilizar Dynamic Scope, se determina siguiendo la pila de llamadas, en este caso, las referencias de “sueño” y “diversión” apuntan a las variables declaradas en “realizar_tarea”, que fue la última subrutina llamada, inicializadas en cinco. Luego el programa tiene una salida distinta, 99+5=104

```
seba@desktop:~/fai/calp/tpo01$ g++ main.cpp && ./a.out
Realizando tarea de dalg...
Sueño: 104
Realizando tarea de calp...
Diversión: 104
```